The Character Streams and Multithreaded Programming

1. Write a Java program to read characters from the keyboard using Reader and display them on the console using Writer.

Program:

```
import java.io.*;

public class ReaderWriterExample {
   public static void main(String[] args) throws IOException {

      Reader reader = new InputStreamReader(System.in);
      Writer writer = new OutputStreamWriter(System.out);

      int ch;

      System.out.println("Enter characters (press Ctrl+Z and Enter to stop):");

      while ((ch = reader.read()) != -1) {
         writer.write(ch);
         writer.flush();
      }

      reader.close();
      writer.close();
   }
}
```

Output

```
C:\Users\ITLAB2.2-62\Desktop>javac ReaderWriterExample.java

C:\Users\ITLAB2.2-62\Desktop>java ReaderWriterExample
Enter characters (press Ctrl+Z and Enter to stop):
^Z

C:\Users\ITLAB2.2-62\Desktop>java ReaderWriterExample
Enter characters (press Ctrl+Z and Enter to stop):
Chandan^Z
Chandan
```

2. Write a Java program to read the contents of a text file using FileReader and display them on the console.

Program:

```
import java.io.*;

public class FileReaderExample {
   public static void main(String[] args) throws IOException {

      FileReader fr = new FileReader("sample.txt");

      int ch;

      while ((ch = fr.read()) != -1) {
         System.out.print((char) ch);
      }

      fr.close();
   }
}
```

Text document:

```
Hello Java
This is FileReader example.
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>dir /b
FileReaderExample.class
FileReaderExample.java
sample.txt

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java FileReaderExample
Hello Java
This is FileReader example.

C:\Users\ITLAB2.2-62\Desktop\25eu02128>
```

3. Write a Java program to write text into a file using FileWriter.

Program:

```
import java.io.*;

public class FileWriterExample {
   public static void main(String[] args) throws IOException {

      FileWriter fw = new FileWriter("output.txt");

      fw.write("Hello Java\n");
      fw.write("This is an example of FileWriter.");

      fw.close();

      System.out.println("Data written successfully.");
   }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac FileWriterExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java FileWriterExample
Data written successfully.

C:\Users\ITLAB2.2-62\Desktop\25eu02128>
```

4. Write a Java program to copy the contents of one text file into another using FileReader and FileWriter.

Program:

```
import java.io.*;

public class FileCopyExample {
   public static void main(String[] args) throws IOException {

      FileReader fr = new FileReader("sample.txt");
      FileWriter fw = new FileWriter("copy.txt");
```

```
        int ch;

        while ((ch = fr.read()) != -1) {
            fw.write(ch);
        }

        fr.close();
        fw.close();

        System.out.println("File copied successfully.");
    }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac FileCopyExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java FileCopyExample
File copied successfully.

C:\Users\ITLAB2.2-62\Desktop\25eu02128>
```

5. Write a Java program to count the number of **characters, words, and lines** in a text file using character streams.

Program:

```
import java.io.*;

public class CountFileExample {
    public static void main(String[] args) throws IOException {

        FileReader fr = new FileReader("sample.txt");

        int ch;
        int characters = 0;
        int words = 0;
        int lines = 0;
        boolean inWord = false;

        while ((ch = fr.read()) != -1) {
```

```
            characters++;

            if (ch == '\n') {
                lines++;
            }

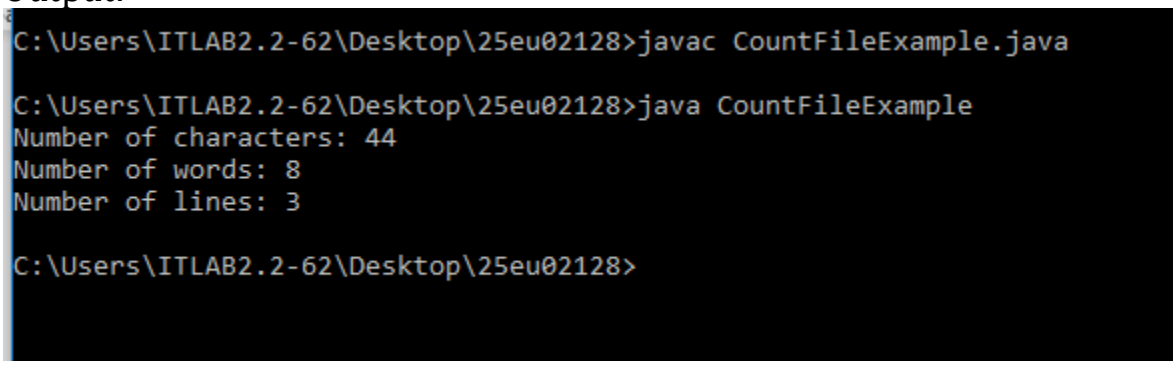
            if (Character.isWhitespace((char) ch)) {
                inWord = false;
            } else if (!inWord) {
                words++;
                inWord = true;
            }
        }

        // Count the last line if the file does not end with a newline
        if (characters > 0) {
            lines++;
        }

        fr.close();

        System.out.println("Number of characters: " + characters);
        System.out.println("Number of words: " + words);
        System.out.println("Number of lines: " + lines);
    }
}
```

Output:



6. Write a Java program to demonstrate the **Main Thread** and display its name, priority, and state.

Program:

```
public class MainThreadExample {
   public static void main(String[] args) {

      // Get the current (main) thread
      Thread t = Thread.currentThread();

      System.out.println("Thread Name: " + t.getName());
      System.out.println("Thread Priority: " + t.getPriority());
      System.out.println("Thread State: " + t.getState());
   }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac MainThreadExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java MainThreadExample
Thread Name: main
Thread Priority: 5
Thread State: RUNNABLE

C:\Users\ITLAB2.2-62\Desktop\25eu02128>
```

7.Write a Java program to create a thread by **extending the** Thread **class**.

Program:

```
class MyThread extends Thread {

   public void run() {
      System.out.println("Thread is running...");
   }
}

public class ThreadExample {
   public static void main(String[] args) {
```

```
        MyThread t = new MyThread();

        t.start();
    }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac ThreadExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java ThreadExample
Thread is running...
```

8. Write a Java program to create a thread by **implementing the** Runnable **interface**.

Program:

```
class MyRunnable implements Runnable {

    public void run() {
        System.out.println("Thread is running...");
    }
}

public class RunnableExample {
    public static void main(String[] args) {

        MyRunnable r = new MyRunnable();

        Thread t = new Thread(r);

        t.start();
    }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac RunnableExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java RunnableExample
Thread is running...
```

9. Write a Java program to create **multiple threads** and demonstrate their concurrent execution.

Program:

```
class MyThread extends Thread {

   private String threadName;

   MyThread(String name) {
      threadName = name;
   }

   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println(threadName + " : " + i);

         try {
            Thread.sleep(500);
         } catch (InterruptedException e) {
            System.out.println(e);
         }
      }
   }
}

public class MultipleThreadsExample {
   public static void main(String[] args) {

      MyThread t1 = new MyThread("Thread 1");
      MyThread t2 = new MyThread("Thread 2");
      MyThread t3 = new MyThread("Thread 3");

      t1.start();
      t2.start();
      t3.start();
   }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac MultipleThreadsExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java MultipleThreadsExample
Thread 1 : 1
Thread 3 : 1
Thread 2 : 1
Thread 3 : 2
Thread 1 : 2
Thread 2 : 2
Thread 3 : 3
Thread 1 : 3
Thread 2 : 3
Thread 3 : 4
Thread 1 : 4
Thread 2 : 4
Thread 3 : 5
Thread 1 : 5
Thread 2 : 5
```

10. Write a Java program to demonstrate the use of isAlive() method to check whether a thread is running.

Program:

```
class MyThread extends Thread {

   public void run() {
      try {
         Thread.sleep(2000);
      } catch (InterruptedException e) {
         System.out.println(e);
      }

      System.out.println("Thread is running...");
   }
}

public class IsAliveExample {
   public static void main(String[] args) {

      MyThread t = new MyThread();

      System.out.println("Before starting: " + t.isAlive());

      t.start();
```

```
        System.out.println("After starting: " + t.isAlive());

        try {
            t.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("After completion: " + t.isAlive());
    }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac IsAliveExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java IsAliveExample
Before starting: false
After starting: true
Thread is running...
After completion: false
```

11. Write a Java program to demonstrate the use of join() method to make one thread wait for another thread to complete.

Program:

```
class MyThread extends Thread {

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Child Thread: " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
```

```
public class JoinExample {
   public static void main(String[] args) {

      MyThread t = new MyThread();

      t.start();

      try {
         t.join();
      } catch (InterruptedException e) {
         System.out.println(e);
      }

      System.out.println("Main Thread completed.");
   }
}
```

Output:

```
C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac JoinExample.java

C:\Users\ITLAB2.2-62\Desktop\25eu02128>java JoinExample
Child Thread: 1
Child Thread: 2
Child Thread: 3
Child Thread: 4
Child Thread: 5
Main Thread completed.
```

12. Write a Java program to create multiple threads and use isAlive() **and** join() to control their execution order.

Program:

```
class MyThread extends Thread {

   private String threadName;

   MyThread(String name) {
      threadName = name;
   }

   public void run() {
```

```
        for (int i = 1; i <= 3; i++) {
            System.out.println(threadName + " : " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class ThreadControlExample {
    public static void main(String[] args) {

        MyThread t1 = new MyThread("Thread 1");
        MyThread t2 = new MyThread("Thread 2");

        t1.start();
        t2.start();

        System.out.println("Thread 1 alive: " + t1.isAlive());
        System.out.println("Thread 2 alive: " + t2.isAlive());

        try {
            // Wait for Thread 1 to complete
            t1.join();

            // After Thread 1 completes, wait for Thread 2
            t2.join();

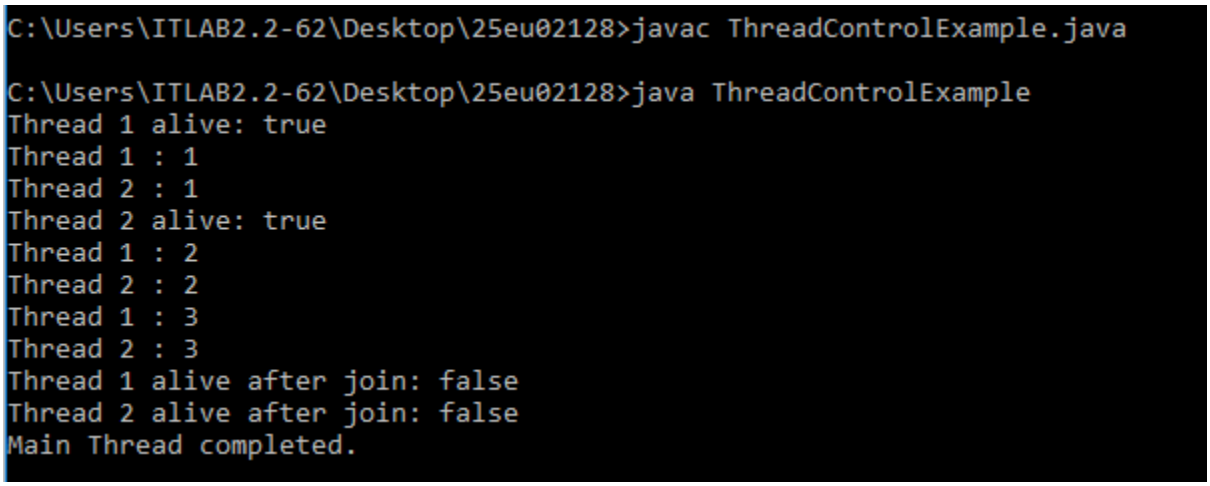
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("Thread 1 alive after join: " + t1.isAlive());
        System.out.println("Thread 2 alive after join: " + t2.isAlive());

        System.out.println("Main Thread completed.");
```

```
    }
}
```

Output:


C:\Users\ITLAB2.2-62\Desktop\25eu02128>javac ThreadControlExample.java
C:\Users\ITLAB2.2-62\Desktop\25eu02128>java ThreadControlExample
Thread 1 alive: true
Thread 1 : 1
Thread 2 : 1
Thread 2 alive: true
Thread 1 : 2
Thread 2 : 2
Thread 1 : 3
Thread 2 : 3
Thread 1 alive after join: false
Thread 2 alive after join: false
Main Thread completed.